

REPORTE DEL PROYECTO II

Araceli Roque Hernández. Inteligencia Artificial, CCC.
Instituto Nacional de Astrofísica, Óptica y Electrónica.
Dr. Manuel Montes y Gómez, Dra. Delia Irazú Hernández Farias

MÉTODOS DE BÚSQUEDA

Este proyecto se centra en la implementación de algoritmos de búsqueda para encontrar una ruta en una cuadrícula. La cuadrícula es de tamaño $N \times N$ y contiene varios obstáculos. El objetivo es mover un robot desde una posición inicial hasta una posición final, evitando los obstáculos.

Se incluye la implementación de tres métodos de búsqueda diferentes: búsqueda en profundidad, algoritmo heurístico (búsqueda haz) y un algoritmo de ruta óptima (A^*).

Búsqueda en profundidad

Una búsqueda en profundidad (DFS) es un algoritmo de búsqueda para lo cual recorre los nodos de un grafo. Su funcionamiento consiste en ir expandiendo cada uno de los nodos que va localizando, de forma recurrente (desde el nodo padre hacia el nodo hijo). Cuando ya no quedan más nodos que visitar en dicho camino, regresa al nodo predecesor, de modo que repite el mismo proceso con cada uno de los vecinos del nodo. Cabe resaltar que, si se encuentra el nodo antes de recorrer todos los nodos, concluye la búsqueda.

Algoritmo heurístico (búsqueda haz)

El algoritmo de Beam Search (BS) es un derivado de un branch-and-bound que utiliza la búsqueda de primero en anchura, e incorpora una heurística para escoger en cada nivel solo los mejores β nodos. Los mejores nodos se guardan en lo que llama el bear, y β suele denominarse bear width. Este método sacrifica completitud a cambio de un enfoque heurístico muy efectivo.

Los mejores β nodos son seleccionados por funciones de evaluación de un paso, las cuales estiman el costo de expandir la solución actual. Nótese que los nodos en el bear representan soluciones parciales, evita el problema de la explosión combinatoria de búsqueda en amplitud por primera vez por ampliar solo los β nodos más prometedores en cada nivel. Una heurística se utiliza para predecir cuáles nodos son propensos a ser el más cerca a la meta.

Ruta óptima (A^*)

A^* es un algoritmo de búsqueda inteligente o informada que busca el camino más corto desde un estado inicial al estado meta a través de un espacio de problema, usando una heurística óptima. Como ignora los pasos más cortos (más "chatos") en algunos casos rinde una solución subóptima.

Un algoritmo de búsqueda admisible es el que garantice el hallazgo de una ruta óptima entre el nodo de inicio y el nodo meta, si es que ella existe. En la búsqueda A^* una heurística admisible

es una que no sobreestima la distancia remanente entre el nodo presente y el nodo meta. Por ejemplo, siempre una ruta real entre dos ciudades es algo mayor y a lo sumo igual a la distancia en línea recta tomada de un mapa. Esta última distancia es así una heurística admisible pues en todo caso es "optimista", lo cual coincide con la definición.

IMPLEMENTACION

La implementación de los métodos de búsqueda se realizó en Python.

Inicialmente se tiene un tablero de tamaño 10 con las posiciones como se muestran en la tabla 1

```
posición_inicial = (0, 0)
posición_final = (9, 9)
matriz[posición_inicial] = 2
matriz[posición_final] = 3
```

Tabla 1

Y los obstáculos que se muestran en la tabla 2

```
obstáculos = [(1, 2), (1, 3), (1, 4), (2, 7), (3, 4), (5, 4), (2, 8), (3, 8),
(4, 2), (5, 3), (7, 3), (7, 4), (6, 0), (7, 0), (8, 0), (9, 0), (7, 5), (7,
8), (8, 8), (9, 4)]
```

Tabla 2

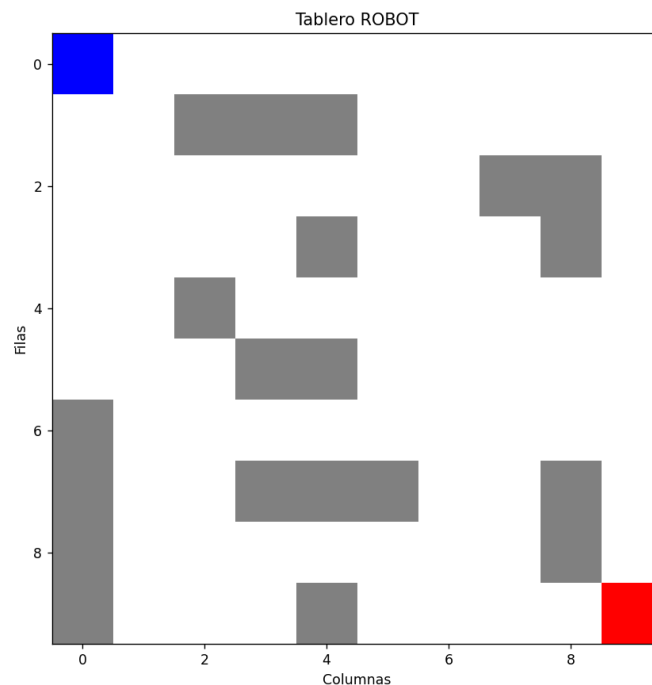


Figura 1. Tablero de tamaño 10 con obstáculos

Búsqueda en profundidad

```
función profundidad
```

Esta función implementa el algoritmo de búsqueda en profundidad (DFS). Toma una posición inicial (`inicio`) y una posición final (`end`) y busca una ruta desde la posición inicial hasta la posición final en el tablero.

Utiliza una pila para llevar un seguimiento de las celdas que se están explorando y comienza con la posición inicial en la pila.

Mientras la pila no esté vacía, el algoritmo saca una celda de la pila, verifica si es la posición final y si no lo es, agrega sus celdas vecinas accesibles a la pila para seguir explorando.

El algoritmo continúa explorando hasta que encuentra la posición final o hasta que la pila esté vacía.

Si encuentra la posición final, devuelve la ruta que lleva desde la posición inicial a la posición final.

Al realizar la búsqueda en profundidad primero se obtuvo la ruta visualizada en la Imagen 2

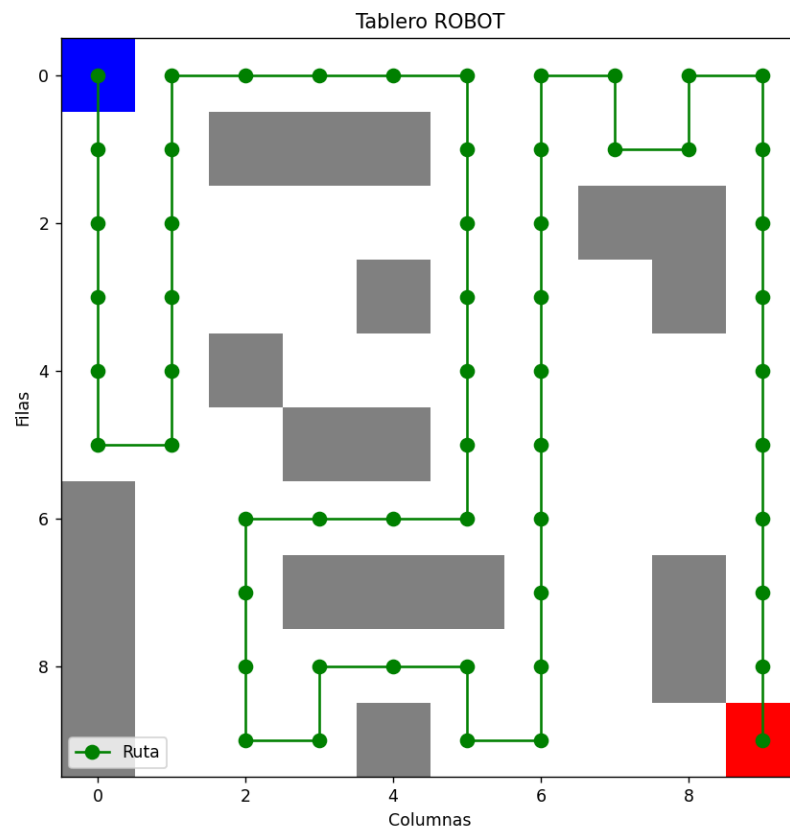


Imagen 2. Tablero con ruta trazada en profundidad

Ruta de búsqueda en profundidad: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0), (5, 1), (4, 1), (3, 1), (2, 1), (1, 1), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (1, 5), (2, 5), (3, 5), (4, 5), (5, 5), (6, 5), (6, 4), (6, 3), (6, 2), (7, 2), (8, 2), (9, 2), (9, 3), (8, 3), (8, 4), (8, 5), (9, 5), (9, 6), (8, 6), (7, 6), (6, 6), (5, 6), (4, 6), (3, 6), (2, 6), (1, 6), (0, 6), (0, 7), (1, 7), (1, 8), (0, 8), (0, 9), (1, 9), (2, 9), (3, 9), (4, 9), (5, 9), (6, 9), (7, 9), (8, 9), (9, 9)]

Búsqueda haz

Función búsqueda_haz

Esta función implementa un algoritmo de búsqueda de haz. Toma una posición inicial (inicio), una posición final (end) y un parámetro opcional `beam_width` (ancho del haz)

Utiliza una cola de prioridad para explorar múltiples caminos a la vez. Comienza con la posición inicial en la cola de prioridad.

El ancho del haz determina cuántos caminos paralelos se explorarán simultáneamente. Por defecto, se establece en 2, lo que significa que se explorarán dos caminos a la vez.

El algoritmo selecciona los caminos con menor costo total (evaluado por la suma de la longitud del camino y una heurística) y continúa explorando hasta que encuentra la posición final o hasta que no queden caminos en el haz.

Si encuentra la posición final, devuelve la ruta que lleva desde la posición inicial a la posición final.

Al realizar la búsqueda haz se obtuvo la ruta visualizada en la Imagen 3

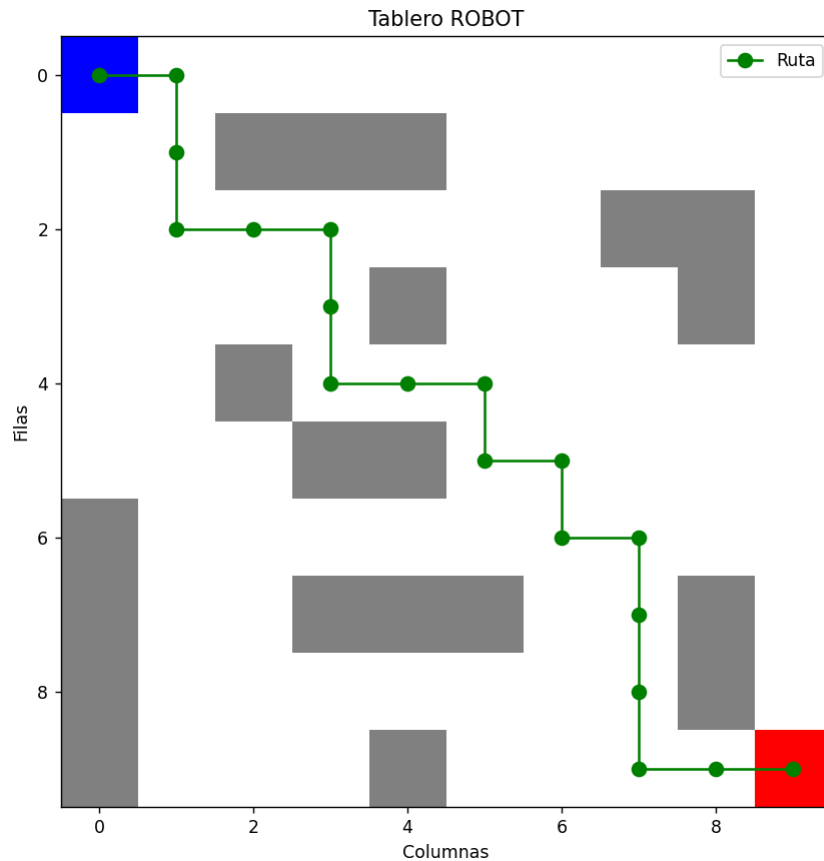


Imagen 3. Tablero con ruta trazada en búsqueda haz

Búsqueda haz: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2), (2, 3), (3, 3), (4, 3), (4, 4), (4, 5), (5, 5), (5, 6), (6, 6), (6, 7), (7, 7), (8, 7), (9, 7), (9, 8), (9, 9)]

Búsqueda A*

Búsqueda_a

Esta función implementa el algoritmo A*. Toma una posición inicial (*inicio*) y una posición final (*final*) y busca una ruta óptima desde la posición inicial hasta la posición final en el tablero.

Utiliza una cola de prioridad para explorar celdas de manera informada.

También utiliza dos diccionarios: *came_from* para hacer un seguimiento de cómo se llega a cada celda y *g_score* para llevar un registro del costo actual para llegar a cada celda desde la posición inicial.

La cola de prioridad prioriza las celdas con un menor costo total (sumando el costo real y una heurística) para explorarlas primero.

Al realizar la búsqueda en profundidad primero se obtuvo la ruta visualizada en la Imagen 4

Ruta A*: [(0, 1), (1, 1), (2, 1), (2, 2), (2, 3), (3, 3), (4, 3), (4, 4), (4, 5), (5, 5), (5, 6), (6, 6), (6, 7), (7, 7), (8, 7), (9, 7), (9, 8), (9, 9)]

CONCLUSION

Los resultados presentados en el proyecto muestran cómo diferentes algoritmos de búsqueda pueden variar en su eficiencia y desempeño al encontrar rutas en una cuadrícula. En particular, se compararon tres algoritmos: Búsqueda en Profundidad, Búsqueda de Haz y Búsqueda A*.

la elección del algoritmo depende de las necesidades específicas de la aplicación. Si se trata de encontrar una solución rápidamente sin preocuparse por la calidad de la ruta, DFS o la búsqueda de haz pueden ser apropiados. Si la calidad de la ruta es fundamental y los costos varían, A* es la mejor opción.

REFERENCIAS

Búsqueda en profundidad—EcuRed. (2001).

https://www.ecured.cu/B%C3%BAsqueda_en_profundidad_numpy.argmaxin—NumPy v1.21 Manual. (2021, junio 22).

<https://numpy.org/doc/stable/reference/generated/numpy.argmaxin.html> parzibyte. (2021, enero 28).

Pila en Python—Estructura de datos. Parzibyte's blog.

<https://parzibyte.me/blog/2021/01/28/pila-python-estructura-datos/>

Todo Sobre Inteligencia Artificial: Métodos de Búsqueda IA. (2013). Todo Sobre Inteligencia Artificial. <https://inteligenciaartificialgrupo33.blogspot.com/p/metodos-de-busqueda-y-ejemplos.html>